

Probability Reminders

Law of Total Probability; $P(x) = \sum_y P(x, y)$

$$\underbrace{X \perp Y}_{\text{X and Y independent}} \Leftrightarrow P(x, y) = P(x) P(y)$$

X and Y independent

n independent binary r.v.'s $\Rightarrow 2^n - 1$ parameters to specify the disto.

$$P(x|y) = \frac{P(x, y)}{P(y)} = \frac{P(y|x) P(x)}{P(y)} \quad (\text{last one can be inferred})$$

Bayes

variables

direct probabilistic relationship

Bayesian Network - DAG consisting of nodes and edges

- Each node X_i is a conditional distribution $P(X_i | Pa(X_i))$
 $P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | Pa(x_i))$

parents of X_i

Conditional Independence $\Leftrightarrow P(x, y | z) = P(x | z) P(y | z)$

$$\Leftrightarrow (x \perp y | z) \Leftrightarrow P(x, y | z) = P(x | z) P(y | z)$$

To check for $(A \perp B | C)$ in a Bayesian network, we need d-separation on all paths between A and B. A path between A and B is d-separated by C if

① The path contains a chain $X \rightarrow C \rightarrow Z$

② The path contains a fork $X \leftarrow C \rightarrow Z$

③ The path contains a v-structure $X \rightarrow Y \leftarrow Z$ s.t. $Y \neq C$ and no descendant of Y is C

Structure Learning

$n = \# \text{ nodes}$

$X_i = \text{discrete r.v.}$
 $r_i = \# \text{ instances of } X_i$

$q_i = \# \text{ parental instances of } X_i$

Bayesian Network Scoring

$$P(G|D) \propto P(G) P(D|G)$$

m_{ijk}
↑ ↑
i-th node j-th parental instantiation k-th node instantiation

$$= P(G) \int P(D|\theta, G) p(\theta|G) d\theta$$

$$= P(G) \prod_{i=1}^n \prod_{j=1}^{q_i} \frac{\Gamma(\alpha_{ij0})}{\Gamma(\alpha_{ij0} + m_{ij0})} \prod_{k=1}^{r_i} \frac{\Gamma(\alpha_{ijk} + m_{ijk})}{\Gamma(\alpha_{ijk})}$$

↓

$$\log P(G|D) = \log P(G) + \sum_{i=1}^n \sum_{j=1}^{q_i} \left(\log \left(\frac{\Gamma(\alpha_{ij0})}{\Gamma(\alpha_{ij0} + m_{ij0})} \right) + \sum_{k=1}^{r_i} \log \left(\frac{\Gamma(\alpha_{ijk} + m_{ijk})}{\Gamma(\alpha_{ijk})} \right) \right)$$

↑
goes away with
uniform prior
 $P(\alpha_{ij0}, \alpha_{ijk} | 1, 1, \dots, 1)$

K2 Search:

$$\alpha_{ij0} = \sum_{k=1}^{r_i} \alpha_{ijk} \quad m_{ij0} = \sum_{k=1}^{r_i} m_{ijk}$$

- Start w/ empty graph
- Iterate over variables in some ordering
 - Greedily add parents to nodes in a way that maximally increases score

Local Search:

- Start with an initial graph and then move to the highest scoring neighbor

Neighbor: one basic graph operation (add, remove, or reverse an edge) away

Strategies to avoid local optima:

Randomized restart: restart the search at a random point in the search space

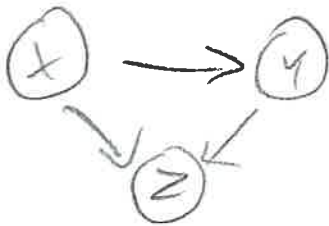
Simulated annealing: visit less fit neighbors (explore) with a random probability.

Markov Equivalent Classes - $A \rightarrow B$ encodes the same independence assumptions as $A \leftarrow B$. If we use a Dirichlet prior such that $k = \frac{1}{2} \sum_{i,j} \theta_{ij}$, all graphs in the same equivalence class get the same score (cutting down the search space)

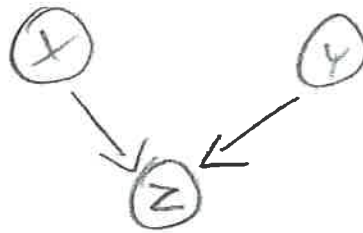
2 graphs are Markov equivalent iff:

- ① Same edges (ignore direction)
- ② Same set of immoral v-structures

Moral



Immoral



Markov Blanket

Minimal set of nodes that if their values were known, make X conditionally independent of all other nodes.

↳ Parents, children, and other parents of its children

Simple Decisions

Constraints on Rational Preferences

- ① Completeness: one of these 3 must hold: $A \succ B$ $A \sim B$ $A \prec B$
- ② Transitivity: $A \succ B \wedge B \succ C \Rightarrow A \succ C$
- ③ Continuity: $A \succ C \succ B \Rightarrow \exists$ probability p such that $[A, p, B, 1-p] \sim C$
- ④ Independence: $A \succ B \Rightarrow \forall C$ and probabilities p , $[A, p, C, 1-p] \succ [B, p, C, 1-p]$

Utility Function:

$$U(A) \succ U(B) \Leftrightarrow A \succ B$$

$$U(A) = U(B) \Leftrightarrow A \sim B$$

Utility of lottery: $U[s_1, p_1, \dots, s_n, p_n] = \sum_i p_i U_i(s_i)$

Maximum Expected Utility Principle:

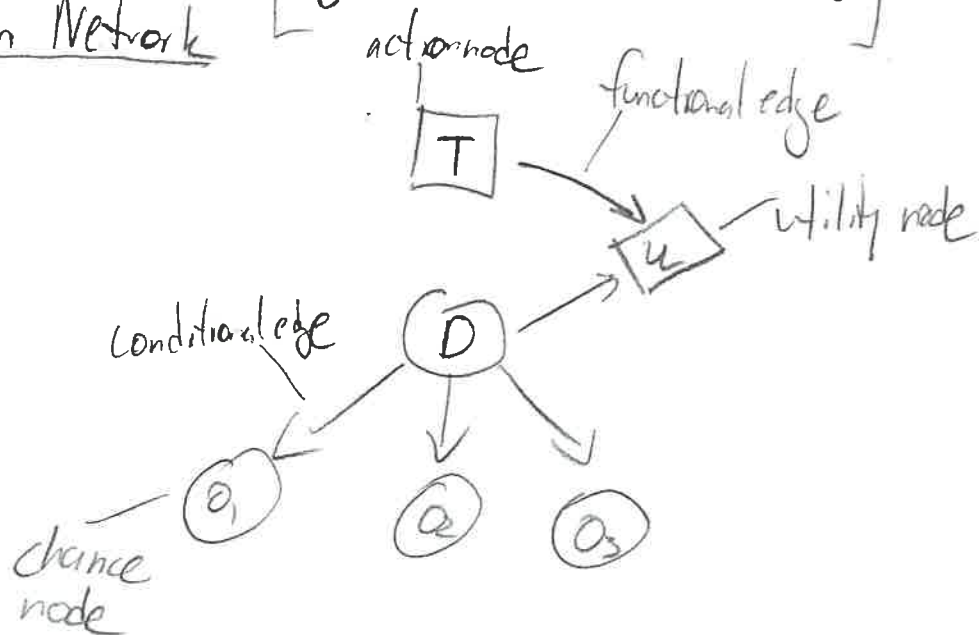
$$E(a|o) = \sum_{s'} P(s'|a, o) U(s')$$

Rational agent chooses $a^* = \arg \max_a EU(a|o)$

Value of Information

$$VOI(o'|o) = \left[\sum_{o'} P(o'|o) EU^*(o, o') \right] - EU^*(o)$$

Decision Network



Value function Policies

$$\pi(s) = \operatorname{argmax}_a \left[R(s, a) + \gamma \sum_{s'} T(s'|s, a) U(s') \right]$$

↳ greedy function wrt U . If U is optimal, then π is optimal.
Action-value function: expected return from starting in state s , taking action a , then continuing with the greedy policy wrt Q ,

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s'|s, a) U(s')$$

Then, $U(s) = \max_a Q(s, a)$

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

Value Iteration

$$U_k(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} T(s'|s, a) U_{k-1}(s') \right]$$

Approximate Value Functions

BIG IDEA: estimate parameters θ to approximate the value function $U_\theta(s)$ typically to many states to store values in a table

Approaches:

① Nearest neighbor - find nearest state (s) s that we do know the value for and use those to estimate

② Kernel smoothing - utilities of states in S are smoothed

$$U_\theta(s) = \theta^T \beta(s), \text{ where } \beta_i(s) = \frac{k(s, s_i)}{\sum_j k(s, s_j)}$$

③ Linear interpolation between 2 states, where grid values are weighted by their distance to the state we are trying to approximate the value of

⑥ Monte Carlo Tree Search (MCTS) - run d simulations. During simulation, update estimates of action-value function $Q(s, a)$ and record the number of times the (s, a) pair has been selected. Use upper confidence bound (UCB) to incentivize exploration.

$$Q(s, a) + C \sqrt{\frac{\log N(s)}{N(s, a)}}$$

Policy search - search the space of policies without directly computing a value function

① Local search (Hooke-Jeeves) - begin with a policy parameterization π_θ and adjust θ by stepping $\pm \alpha$ in a single coordinate direction that best improves the policy

② Cross entropy method

$$\theta \sim p(\theta | \psi)$$

↑ parameters of dist.

• Sample $\theta \sim p(\theta | \psi)$

• find the elite sample θ 's

• fit a distribution of elite samples

↻ loop

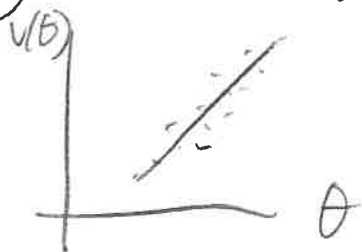
Policy Gradient Estimation

① Finite difference

(i.e. linear approximation)

$$\frac{U(\theta + \delta e^{(i)}) - U(\theta)}{\delta}$$

② Regression gradient



From many rollouts, run regression. The slope approximates the gradient.

Offline Belief State Planning

Fully Observable Value Approximation

QMDP: $\alpha_a^{(h+1)}(s) = R(s, a) + \gamma \sum_{s'} T(s'|s, a) \max_a \alpha_a^{(h)}(s')$

Fast informed bound: $\alpha_a^{(h+1)}(s) = R(s, a) + \gamma \sum_o \max_a \sum_{s'} O(o|a, s') T(s'|s, a) \alpha_a^{(h)}(s')$

↳ can be used to produce upper bounds on the value function

Fast Lower Bounds

Best action, worst state: $\alpha_{\text{baws}} = \begin{bmatrix} r_{\text{baws}} \\ \vdots \\ r_{\text{baws}} \end{bmatrix}$
$$r_{\text{baws}} = \max_a \sum_{h=1}^{\infty} \gamma^{h-1} \min_s R(s, a) = \frac{1}{1-\gamma} \max_a \min_s R(s, a)$$

Blind lower bound: $\alpha_a^{(h+1)}(s) = R(s, a) + \gamma \sum_{s'} T(s'|s, a) \alpha_a^{(h)}(s')$

↳ start with α_{baws} , then update

Point-Based Value Iteration (converges to lower bound)

- ① Initialize α vectors in Γ to a lower bound on the value function
- ② Iterate through each belief point and perform a backup.

Given belief point b and alpha vectors Γ :

For each possible action a :

For each possible observation o :

$$\alpha_{a,o} = \operatorname{argmax}_{a \in \Gamma} \alpha^T \text{Update}(b, a, o)$$

$$\alpha_a(s) = R(s, a) + \gamma \sum_{s', o} O(o|a, s') T(s'|s, a) \alpha_{a,o}(s')$$

$$\alpha = \operatorname{argmax}_{\alpha_a} \alpha_a^T b$$

Beliefs

Discrete State Filter

$O(o|a, s') :=$ probability of observing o given action a , new state s'

An POMDP, agent does not know its state, but rather keeps a belief $b(s)$, the probability it is in state s .

Belief update.

$$b'(s') \propto O(o|a, s') \sum_s T(s'|s, a) b(s)$$

Particle Filter

- ① Sample a collection of states $\rightarrow$ initial beliefs
- ② Propagate these states to obtain s'_i with probability $T(s'_i|s_i, a)$
- ③ Weight states according to observation function $w_i = O(o|a, s'_i)$

Exact POMDP Solution Methods

Belief-State MDPs - turn belief into a state

$\hookrightarrow$ can be challenging since $b(s)$ is continuous on $[0, 1]$, which causes the resulting state space to be continuous

$$R(b, a) = \sum_s R(s, a) b(s)$$

$$T(b'|b, a) = \sum_o (b' = \text{Update}(b, a, o)) \sum_{s'} O(o|a, s') \sum_s T(s'|s, a) b(s)$$

Conditional Plans - what to do in response to our observations up to a certain depth

- For plan π , the action associated with the root is denoted as $\pi(c)$
- Action associated with observation o is denoted as $\pi(o)$

Online Belief Planning - determine the optimal policy by planning from the current belief. 4 algorithms (review from Ch. 9)

- ① Lookahead with rollouts
- ② Forward search
- ③ Branch and bound
- ④ Monte Carlo Tree Search